

Resolución del Práctico 5

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Criterios de cobertura lógica	3
2. Ejercicio 1: cinco predicados	3
2.1. $p = a \vee (b \wedge c)$	3
2.2. $p = a \rightarrow (b \rightarrow c)$	4
2.3. $p = (a \vee b) \wedge (c \vee d)$	5
2.4. $p = a \oplus b$	6
2.5. $p = a \vee b \vee (c \wedge d)$	6
3. Ejercicio 2: <code>checkIt</code> vs <code>checkItExpand</code>	8
3.1. (a) Expandir a <code>checkItExpand</code> con instrumentación	8
3.2. (b) Derivar T1 (CACC) y T2 (Edge Coverage)	9
3.3. (c) Implementación JUnit	9
3.4. Ejecución	10
4. Ejercicio 3: <code>Thermostat.turnHeaterOn()</code>	10
4.1. Predicado principal y cláusulas	10
4.2. Setup común	10
4.3. (a) CC sin PC	10
4.4. (b) PC sin CC	11
4.5. (c) CACC	11
4.6. Ejecución	11
5. Ejercicio 4: <code>TriTyp.triang()</code>	11
5.1. Predicados considerados	11
5.2. Suite	12
5.3. Pares CACC por predicado	12
5.4. Verificación en código	13

5.5. Ejecución	13
--------------------------	----

1. Criterios de cobertura lógica

El TP5 trabaja sobre **testing basado en expresiones lógicas**. Los criterios que se usan en todos los ejercicios son:

- **PC** (Predicate Coverage): cada predicado p se evalúa a *true* y a *false*.
- **CC** (Clause Coverage): cada cláusula c se evalúa a *true* y a *false*.
- **CACC** (Correlated Active Clause Coverage): para cada cláusula mayor c_i existe un par de tests donde c_i *determina* el predicado y, en ese par, p cambia de valor.
- **RACC** (Restricted Active Clause Coverage): igual que CACC, pero además las cláusulas menores tienen *exactamente los mismos valores* en ambos tests del par.

Determinación de una cláusula. c_i determina a p cuando, fijando las cláusulas menores, el valor de p depende de c_i . Operativamente: si al cambiar c_i y dejar fijas las demás, p cambia, entonces c_i determina p .

2. Ejercicio 1: cinco predicados

Para cada predicado, los incisos (a) cláusulas, (b) tabla de verdad, (c) CC pero no PC, (d) PC pero no CC, (e) pares CACC, (f) pares RACC.

Convención: las filas se numeran en orden lexicográfico con T antes que F; los pares se listan como $(\text{fila}_i, \text{fila}_j)$.

2.1. $p = a \vee (b \wedge c)$

(a) Cláusulas: a , b , c .

(b) Tabla de verdad:

Fila	a	b	c	p
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	T
5	F	T	T	T
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

(c) CC pero no PC: {fila 4, fila 5} (ambas dan $p = T$, pero las cláusulas toman ambos valores entre las dos).

(d) PC pero no CC: {fila 1, fila 6}.

(e) Pares CACC:

- mayor a : (2, 6), (2, 7), (2, 8), (3, 6), (3, 7), (3, 8), (4, 6), (4, 7), (4, 8)
- mayor b : (5, 7)
- mayor c : (5, 6)

(f) Pares RACC:

- mayor a : (2, 6), (3, 7), (4, 8)
- mayor b : (5, 7)
- mayor c : (5, 6)

2.2. $p = a \rightarrow (b \rightarrow c)$

(a) Cláusulas: a , b , c .

(b) Tabla de verdad:

Fila	a	b	c	p
1	T	T	T	T
2	T	T	F	F
3	T	F	T	T
4	T	F	F	T
5	F	T	T	T
6	F	T	F	T
7	F	F	T	T
8	F	F	F	T

(c) CC pero no PC: {fila 1, fila 8}.

(d) PC pero no CC: {fila 1, fila 2}.

(e) Pares CACC:

- mayor a : (2, 6)
- mayor b : (2, 4)
- mayor c : (1, 2)

(f) Pares RACC: idénticos a CACC.

2.3. $p = (a \vee b) \wedge (c \vee d)$

(a) Cláusulas: a, b, c, d .

(b) Tabla de verdad (16 filas):

Fila	a	b	c	d	p
1	T	T	T	T	T
2	T	T	T	F	T
3	T	T	F	T	T
4	T	T	F	F	F
5	T	F	T	T	T
6	T	F	T	F	T
7	T	F	F	T	T
8	T	F	F	F	F
9	F	T	T	T	T
10	F	T	T	F	T
11	F	T	F	T	T
12	F	T	F	F	F
13	F	F	T	T	F
14	F	F	T	F	F
15	F	F	F	T	F
16	F	F	F	F	F

(c) CC pero no PC: {fila 4, fila 13}.

(d) PC pero no CC: {fila 1, fila 4}.

(e) Pares CACC:

- mayor a : (5, 13), (5, 14), (5, 15), (6, 13), (6, 14), (6, 15), (7, 13), (7, 14), (7, 15)
- mayor b : (9, 13), (9, 14), (9, 15), (10, 13), (10, 14), (10, 15), (11, 13), (11, 14), (11, 15)
- mayor c : (2, 4), (2, 8), (2, 12), (4, 6), (4, 10), (6, 8), (6, 12), (8, 10), (10, 12)
- mayor d : (3, 4), (3, 8), (3, 12), (4, 7), (4, 11), (7, 8), (7, 12), (8, 11), (11, 12)

(f) Pares RACC:

- mayor a : (5, 13), (6, 14), (7, 15)
- mayor b : (9, 13), (10, 14), (11, 15)

- mayor c : $(2, 4), (6, 8), (10, 12)$
- mayor d : $(3, 4), (7, 8), (11, 12)$

2.4. $p = a \oplus b$

(a) Cláusulas: a, b .

(b) Tabla de verdad:

Fila	a	b	p
1	T	T	F
2	T	F	T
3	F	T	T
4	F	F	F

(c) CC pero no PC: {fila 1, fila 4}.

(d) PC pero no CC: {fila 1, fila 2}.

(e) Pares CACC:

- mayor a : $(1, 3), (2, 4)$
- mayor b : $(1, 2), (3, 4)$

(f) Pares RACC: idénticos a CACC.

2.5. $p = a \vee b \vee (c \wedge d)$

(a) Cláusulas: a, b, c, d .

(b) Tabla de verdad (16 filas):

Fila	a	b	c	d	p
1	T	T	T	T	T
2	T	T	T	F	T
3	T	T	F	T	T
4	T	T	F	F	T
5	T	F	T	T	T
6	T	F	T	F	T
7	T	F	F	T	T
8	T	F	F	F	T
9	F	T	T	T	T
10	F	T	T	F	T
11	F	T	F	T	T
12	F	T	F	F	T
13	F	F	T	T	T
14	F	F	T	F	F
15	F	F	F	T	F
16	F	F	F	F	F

(c) CC pero no PC: {fila 4, fila 13}.

(d) PC pero no CC: {fila 1, fila 14}.

(e) Pares CACC:

- mayor a : (6, 14), (6, 15), (6, 16), (7, 14), (7, 15), (7, 16), (8, 14), (8, 15), (8, 16)
- mayor b : (10, 14), (10, 15), (10, 16), (11, 14), (11, 15), (11, 16), (12, 14), (12, 15), (12, 16)
- mayor c : (13, 15)
- mayor d : (13, 14)

(f) Pares RACC:

- mayor a : (6, 14), (7, 15), (8, 16)
- mayor b : (10, 14), (11, 15), (12, 16)
- mayor c : (13, 15)
- mayor d : (13, 14)

3. Ejercicio 2: checkIt vs checkItExpand

Trabajo sobre el predicado del método `checkIt()`:

$$p = a \wedge (b \vee c).$$

3.1. (a) Expandir a `checkItExpand` con instrumentación

La transformación pedida busca que cada decisión del `if` use una sola variable booleana.

Versión original:

```
1 public static boolean checkIt(boolean a, boolean b, boolean c) {  
2     return a && (b || c);  
3 }
```

Versión expandida (`checkItExpand`) con la misma semántica:

```
1 public static ExecutionTrace checkItExpand(boolean a, boolean b,  
2     boolean c) {  
3     List<String> nodes = new ArrayList<String>();  
4     nodes.add("START");  
5  
6     boolean result;  
7     nodes.add("A");  
8     if (a) {  
9         nodes.add("B");  
10        if (b) {  
11            nodes.add("TRUE"); result = true;  
12        } else {  
13            nodes.add("C");  
14            if (c) { nodes.add("TRUE"); result = true; }  
15            else { nodes.add("FALSE"); result = false; }  
16        }  
17    } else {  
18        nodes.add("FALSE"); result = false;  
19    }  
20    nodes.add("END");  
21    return new ExecutionTrace(result, nodes);  
22 }
```

`ExecutionTrace` guarda los nodos recorridos y permite consultar las aristas como pares consecutivos $N_i \rightarrow N_{i+1}$. Los nodos del CFG son: START, A, B, C, TRUE, FALSE, END.

3.2. (b) Derivar T1 (CACC) y T2 (Edge Coverage)

T1 para CACC sobre `checkIt`

Suite propuesta:

$$t_1 = (T, T, F), \quad t_2 = (F, T, F), \quad t_3 = (T, F, F), \quad t_4 = (T, F, T).$$

- Mayor a : par (t_1, t_2) con menores $b = T, c = F$ fijas; p pasa de T a F .
- Mayor b : par (t_1, t_3) con menores $a = T, c = F$; p pasa de T a F .
- Mayor c : par (t_4, t_3) con menores $a = T, b = F$; p pasa de T a F .

T1 satisface CACC.

T2 para Edge Coverage sobre `checkItExpand`

Suite propuesta:

$$e_1 = (F, F, F), \quad e_2 = (T, T, F), \quad e_3 = (T, F, T), \quad e_4 = (T, F, F).$$

Aristas del CFG expandido (9):

$$\begin{aligned} & \text{START} \rightarrow A, \quad A \rightarrow B, \quad A \rightarrow \text{FALSE}, \quad B \rightarrow \text{TRUE}, \quad B \rightarrow C, \\ & C \rightarrow \text{TRUE}, \quad C \rightarrow \text{FALSE}, \quad \text{TRUE} \rightarrow \text{END}, \quad \text{FALSE} \rightarrow \text{END}. \end{aligned}$$

$e_1 \dots e_4$ cubren las 9 aristas, así que T2 satisface Edge Coverage.

¿T2 satisface CACC sobre `checkIt`?

No. Para que a determine p hace falta $b \vee c = T$. En T2 hay casos con $a = T$ y $b \vee c = T$ (e_2, e_3), pero no hay ningún caso con $a = F$ y $b \vee c = T$. Falta el par correlacionado para la cláusula mayor a , así que T2 no satisface CACC.

3.3. (c) Implementación JUnit

Las suites están en `CheckItT1CaccTest` (T1) y `CheckItT2EdgeCoverageTest` (T2). Ejemplo del test para la cláusula mayor a :

```
1 @Test
2 public void t1ContainsAValidCaccPairForMajorA() {
3     boolean pWhenATrue = CheckIt.checkIt(true, true, false);
4     boolean pWhenAFalse = CheckIt.checkIt(false, true, false);
5     assertTrue(pWhenATrue);
```

```
6     assertFalse(pWhenAFalse);  
7 }
```

3.4. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true  
-Dtest=CheckItT1CaccTest,CheckItT2EdgeCoverageTest test
```

Resultado: BUILD SUCCESS, Tests run: 5, Failures: 0, Errors: 0.

4. Ejercicio 3: `Thermostat.turnHeaterOn()`

Tests para `turnHeaterOn()` que cumplan tres criterios distintos.

4.1. Predicado principal y cláusulas

$$p = (a \vee (b \wedge c)) \wedge d.$$

Cláusulas:

- a : `curTemp < dTemp - thresholdDiff`
- b : `override`
- c : `curTemp < overTemp - thresholdDiff`
- d : `timeSinceLastRun > minLag`

4.2. Setup común

`dTemp` es interno del objeto. En todos los tests fijé:

- `setSetting(MORNING, WEEKDAY, 69)`
- `setPeriod(MORNING), setDay(WEEKDAY)`
- `thresholdDiff = 5, minLag = 10`

El setup vive en `ThermostatCoverageSupport`.

4.3. (a) CC sin PC

Sí se puede.

- $t_1 = (a = T, b = F, c = F, d = F) \rightarrow p = F$
- $t_2 = (a = F, b = T, c = F, d = T) \rightarrow p = F$
- $t_3 = (a = F, b = F, c = T, d = T) \rightarrow p = F$

CC se cumple porque cada cláusula toma T y F al menos una vez. PC no se cumple: p es siempre F .

4.4. (b) PC sin CC

Sí se puede.

- $u_1 = (T, T, T, T) \rightarrow p = T$
- $u_2 = (T, T, T, F) \rightarrow p = F$

PC se cumple. CC no: a , b y c nunca toman F .

4.5. (c) CACC

Pares por cláusula mayor:

Mayor	Par	p
a	(T, F, F, T) vs (F, F, F, T)	$T \rightarrow F$
b	(F, T, T, T) vs (F, F, T, T)	$T \rightarrow F$
c	(F, T, T, T) vs (F, T, F, T)	$T \rightarrow F$
d	(T, F, F, T) vs (T, F, F, F)	$T \rightarrow F$

La suite final queda en 6 tests únicos porque varios casos se reutilizan entre pares.

4.6. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true
-Dtest=ThermostatExercise3*Test test
```

Resultado: 11 tests entre `checkit` y `thermostat`, sin fallas.

5. Ejercicio 4: `TriTyp.triang()`

Tests JUnit para `TriTyp.triang()` que satisfacen CACC.

5.1. Predicados considerados

1. $P_1: (s_1 \leq 0) \vee (s_2 \leq 0) \vee (s_3 \leq 0)$

2. $P_2: s_1 = s_2$
3. $P_3: s_1 = s_3$
4. $P_4: s_2 = s_3$
5. $P_5: \text{triOut} = 0$
6. $P_6: (s_1 + s_2 \leq s_3) \vee (s_2 + s_3 \leq s_1) \vee (s_1 + s_3 \leq s_2)$
7. $P_7: \text{triOut} > 3$
8. $P_8: (\text{triOut} = 1) \wedge (s_1 + s_2 > s_3)$
9. $P_9: (\text{triOut} = 2) \wedge (s_1 + s_3 > s_2)$
10. $P_{10}: (\text{triOut} = 3) \wedge (s_2 + s_3 > s_1)$

Para CACC: en los predicados con varias cláusulas ($P_1, P_6, P_8, P_9, P_{10}$) verifico pares por cláusula mayor; en los de una sola cláusula (P_2, P_3, P_4, P_5, P_7) alcanza con cubrir *true* y *false*.

5.2. Suite

Casos (s_1, s_2, s_3) :

- $(-1, 1, 1), (1, -1, 1), (1, 1, -1), (1, 1, 1)$
- $(1, 2, 3), (2, 3, 4), (3, 1, 2), (1, 3, 2)$
- $(1, 2, 1), (2, 2, 1), (1, 1, 2), (2, 1, 2), (1, 2, 2), (2, 1, 1)$

5.3. Pares CACC por predicado

P_1 :

- mayor c_1 : $(-1, 1, 1)$ vs $(1, 1, 1)$
- mayor c_2 : $(1, -1, 1)$ vs $(1, 1, 1)$
- mayor c_3 : $(1, 1, -1)$ vs $(1, 1, 1)$

P_6 :

- mayor d_1 : $(1, 2, 3)$ vs $(2, 3, 4)$
- mayor d_2 : $(3, 1, 2)$ vs $(2, 3, 4)$
- mayor d_3 : $(1, 3, 2)$ vs $(2, 3, 4)$

P_8 :

- mayor e_1 : (1, 2, 1) vs (2, 2, 1)
- mayor e_2 : (2, 2, 1) vs (1, 1, 2)

P_9 :

- mayor f_1 : (1, 1, 2) vs (2, 1, 2)
- mayor f_2 : (1, 2, 1) vs (2, 1, 2)

P_{10} :

- mayor g_1 : (1, 1, 2) vs (1, 2, 2)
- mayor g_2 : (2, 1, 1) vs (1, 2, 2)

La misma suite cubre *true/false* para P_2 , P_3 , P_4 , P_5 y P_7 .

5.4. Verificación en código

TriTypTraceSupport instrumenta los predicados y cláusulas en los tests, y además valida que la traza dé la misma salida que `TriTyp.triang()`. Eso evita que el modelo de trazas se despegue del código real.

5.5. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true
    -Dtest=TriTypExercise4CaccTest test
```